

Slip 1

Q 2) Consider the following database(primary keys are underlined).

Room (roomno,room_name,room_type,charges)

Guest(Guestcode, Gname,city)

The relationship is as follows: Room-Guest: one-to-one. room_type can have values as either 'AC' or 'NonAC'.

- A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]**
- B. Execute the following queries in PostGreSQL (any 3) [6 marks]**
- i. List the details of the rooms having charges between 5000 and 10000.**
 - ii. List the names of the guests in the sorted order by city name.**
 - iii. List the minimum charges of a room.**
 - iv. Increase the charges of all AC rooms by 15%.**
 - v. List the names of all the NONAC rooms whose charges are more than 10000.**
- C. Write a query to list the name of the guest to whom the room with highest charges is allotted. [4 marks]**

OR

- C. Create a view to list the names of all the NonAC rooms which have charges greater than at least one of the 'AC' room.**

```
-- Create Room table
CREATE TABLE Room (
    roomno INT PRIMARY KEY,
    room_name VARCHAR(50),
    room_type VARCHAR(10) CHECK (room_type IN ('AC', 'NonAC')),
    charges NUMERIC(10,2)
);

-- Create Guest table
CREATE TABLE Guest (
    Guestcode INT PRIMARY KEY,
    Gname VARCHAR(50),
    city VARCHAR(50),
    roomno INT UNIQUE, -- one-to-one relationship
    FOREIGN KEY (roomno) REFERENCES Room(roomno)
);
```

-- Insert sample records into Room

INSERT INTO Room VALUES

(1, 'Deluxe1', 'AC', 8000),
(2, 'Deluxe2', 'NonAC', 6000),
(3, 'Suite1', 'AC', 12000),
(4, 'Suite2', 'NonAC', 15000);

-- Insert sample records into Guest

INSERT INTO Guest VALUES

(101, 'Ravi', 'Pune', 1),
(102, 'Meena', 'Mumbai', 2),
(103, 'Arjun', 'Delhi', 3),
(104, 'Kiran', 'Bangalore', 4);

B. Queries (any 3)

i. List details of rooms with charges between 5000 and 10000

SELECT *
FROM Room
WHERE charges BETWEEN 5000 AND 10000;

ii. List names of guests in sorted order by city

SELECT Gname, city
FROM Guest
ORDER BY city;

iii. List the minimum charges of a room

SELECT MIN(charges) AS min_charges
FROM Room;

iv. Increase charges of all AC rooms by 15%

```
UPDATE Room
SET charges = charges * 1.15
WHERE room_type = 'AC';
```

v. List names of all NonAC rooms with charges > 10000

```
SELECT room_name
FROM Room
WHERE room_type = 'NonAC' AND charges > 10000;
```

C. Query for highest charges room's guest

```
SELECT Gname
FROM Guest
WHERE roomno = (
    SELECT roomno
    FROM Room
    WHERE charges = (SELECT MAX(charges) FROM Room)
);
```

Alternative C. Create a View

```
CREATE VIEW NonAC_View AS
SELECT room_name, charges
FROM Room
WHERE room_type = 'NonAC'
AND charges > ANY (SELECT charges FROM Room WHERE room_type = 'AC');
```

Slip 2

Q 2) Consider the following database(primary keys are underlined).

Employee(eno,ename,designation,salary)

Department(dno,dname,location)

The relationship is as follows: Employee-Department: many-to-one.

- A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]**
- B. Execute the following queries in PostGreSQL (any 3) [6 marks]**
- i. Give a 5% raise in salary to all the employees.**
 - ii. Display average salary of an employee.**
 - iii. List the details of all the departments located at 'Nashik' city.**
 - iv. Display the details of employees whose names ends with an alphabet 'a'.**
 - v. Display the location of 'HR' department.**

C. Write a query to list the details of employees who do not work in any of the departments located at 'Delhi'. [4 marks] OR

C. Create a view to list the names of employees whose salary is greater than all the employees working in 'HR' department.

-- Create Department table

```
CREATE TABLE Department (  
    dno INT PRIMARY KEY,  
    dname VARCHAR(50),  
    location VARCHAR(50)  
);
```

-- Create Employee table (many-to-one: each employee belongs to one department)

```
CREATE TABLE Employee (  
    eno INT PRIMARY KEY,  
    ename VARCHAR(50),  
    designation VARCHAR(50),  
    salary NUMERIC(10,2),  
    dno INT,  
    FOREIGN KEY (dno) REFERENCES Department(dno)  
);
```

-- Insert sample Department records

```
INSERT INTO Department VALUES  
(1, 'HR', 'Delhi'),  
(2, 'Finance', 'Nashik'),  
(3, 'IT', 'Mumbai'),  
(4, 'Admin', 'Nashik');
```

-- Insert sample Employee records

```
INSERT INTO Employee VALUES
```

```
(101, 'Ravi', 'Manager', 50000, 1),  
(102, 'Meena', 'Clerk', 25000, 2),  
(103, 'Aruna', 'Analyst', 30000, 2),  
(104, 'Kiran', 'Developer', 45000, 3),  
(105, 'Sonia', 'HR Executive', 28000, 1);
```

B. Queries (any 3)

i. Give a 5% raise in salary to all employees

```
UPDATE Employee  
SET salary = salary * 1.05;
```

ii. Display average salary of employees

```
SELECT AVG(salary) AS avg_salary  
FROM Employee;
```

iii. List details of all departments located at 'Nashik'

```
SELECT *  
FROM Department  
WHERE location = 'Nashik';
```

iv. Display employees whose names end with 'a'

```
SELECT *  
FROM Employee  
WHERE ename LIKE '%a';
```

v. Display the location of 'HR' department

```
SELECT location  
FROM Department  
WHERE dname = 'HR';
```

C. Choice

1. Employees not working in departments located at 'Delhi'

```
SELECT *  
FROM Employee  
WHERE dno NOT IN (  
    SELECT dno  
    FROM Department  
    WHERE location = 'Delhi'  
);
```

2. Create a view: employees with salary greater than all employees in HR

```
CREATE VIEW HighSalaryEmployees AS  
SELECT ename, salary  
FROM Employee  
WHERE salary > ALL (  
    SELECT salary  
    FROM Employee E  
    JOIN Department D ON E.dno = D.dno  
    WHERE D.dname = 'HR'  
);
```

OR

```
CREATE VIEW HighSalaryEmployees AS  
SELECT ename, salary  
FROM Employee  
WHERE salary > ALL (  
    SELECT salary  
    FROM Employee  
    WHERE dno = (  
        SELECT dno  
        FROM Department  
        WHERE dname = 'HR'  
    )  
);
```

Slip 3

**Q 2) Consider the following database (primary keys are underlined).
Person (pnumber, pname, birthdate, income)**

Area (aname, area_type, pincode)

The relationship is as follows: Person-Area: many-to-one. The 'area_type' can have values as either 'urban' or 'rural'.

A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostgreSQL (any 3) [6 marks]

- i. List the details of all people whose name starts with the alphabet 'R'.**

- ii. List the names of all people whose birthday falls in the month of 'July'.**

- iii. Display the details of people in the sorted order of their income.**

- iv. Display the count of areas of 'urban' type.**

- v. Change the pincode of 'kalyaninagar' to 411036.**

C. Write a query to list the names of people who live in 'Camp' area and have income less than at least one person who lives in 'kalyaninagar' area. [4 marks]

OR

C. Create a view to list the details of the person with second maximum income.

-- Create Area table

CREATE TABLE Area (

aname VARCHAR(50) PRIMARY KEY,

area_type VARCHAR(10) CHECK (area_type IN ('urban', 'rural')),

pincode INT

);

-- Create Person table

CREATE TABLE Person (

```
pnumber INT PRIMARY KEY,  
  
pname VARCHAR(50),  
  
birthdate DATE,  
  
income NUMERIC(10,2),  
  
aname VARCHAR(50),  
  
FOREIGN KEY (aname) REFERENCES Area(aname)  
  
);
```

-- Insert sample records into Area

```
INSERT INTO Area VALUES  
  
('Camp', 'urban', 411001),  
  
('Kalyaninagar', 'urban', 411014),  
  
('Hadapsar', 'rural', 412308),  
  
('ShivajiNagar', 'urban', 411005);
```

-- Insert sample records into Person

```
INSERT INTO Person VALUES  
  
(1, 'Ravi', '1995-07-12', 45000, 'Camp'),  
  
(2, 'Meena', '1998-03-25', 30000, 'Kalyaninagar'),  
  
(3, 'Raj', '1992-07-05', 60000, 'Hadapsar'),
```


(4, 'Kiran', '1990-11-18', 70000, 'ShivajiNagar'),

(5, 'Rohit', '1999-01-09', 25000, 'Camp');

B. Queries (any 3)

i. List details of people whose name starts with 'R'

```
SELECT *
```

```
FROM Person
```

```
WHERE pname LIKE 'R%';
```

ii. List names of people whose birthday is in July

```
SELECT pname, birthdate
```

```
FROM Person
```

```
WHERE EXTRACT(MONTH FROM birthdate) = 7;
```

iii. Display details of people sorted by income

```
SELECT *
```

```
FROM Person
```

```
ORDER BY income;
```

iv. Count areas of 'urban' type

```
SELECT COUNT(*) AS urban_count
```

```
FROM Area
```

```
WHERE area_type = 'urban';
```

v. Change pincode of 'Kalyaninagar' to 411036

```
UPDATE Area
```

```
SET pincode = 411036
```

```
WHERE aname = 'Kalyaninagar';
```

C. Choice

1. People in 'Camp' with income < at least one person in

'Kalyaninagar'

```
SELECT pname
```

```
FROM Person
```

```
WHERE aname = 'Camp'

AND income < ANY (

    SELECT income

    FROM Person

    WHERE aname = 'Kalyaninagar'

);
```

2. Create a view for the person with second maximum income

```
CREATE VIEW SecondMaxIncome AS

SELECT *

FROM Person

WHERE income = (

    SELECT MAX(income)

    FROM Person

    WHERE income < (SELECT MAX(income) FROM Person)

);
```

Slip 4

Q.2) Consider the following database (primary keys are underlined).

Policy (pno, pname,premium_amt,policy_type)

Customer (cno,cname.city, agent_name)

The relationship is as follows: Policy-Customer: many-to-one. The 'policy_type' can have values as 'Yearly', 'Half-yearly' or 'Monthly'.

A. Create the above database in PostGreSQL and insert sufficient records. [5 marks] B. Execute the following queries in PostgreSQL (any 3) [6 marks]

- i. List the details of all customers who live in city.**
- ii. Display the average premium amount.**
- iii. Count the number of customers who have taken 'Jeevan Anand' policy.**
- iv. Increase the premium amount for 'Monthly' policies by 10%.**
- v. Display the policy_type wise count of policies.**

C. Write a query to list the policy_type with highest average premium amount.

[4 marks]

OR

C. Create a view to list the name of the customer who has taken highest number of policies.

-- Create Policy table

CREATE TABLE Policy (

pno INT PRIMARY KEY,

pname VARCHAR(50),

premium_amt NUMERIC(10,2),

policy_type VARCHAR(20) CHECK (policy_type IN ('Yearly', 'Half-yearly', 'Monthly'))

);

-- Create Customer table (many-to-one: each customer is linked to one policy)

```
CREATE TABLE Customer (  
    cno INT PRIMARY KEY,  
    cname VARCHAR(50),  
    city VARCHAR(50),  
    agent_name VARCHAR(50),  
    pno INT,  
    FOREIGN KEY (pno) REFERENCES Policy(pno)  
);
```

-- Insert sample records into Policy

```
INSERT INTO Policy VALUES  
  
(1, 'Jeevan Anand', 25000, 'Yearly'),  
  
(2, 'Health Plus', 12000, 'Half-yearly'),  
  
(3, 'Child Secure', 2000, 'Monthly'),  
  
(4, 'Life Secure', 5000, 'Monthly'),  
  
(5, 'Retirement Plan', 30000, 'Yearly');
```

-- Insert sample records into Customer

INSERT INTO Customer VALUES

(101, 'Ravi', 'Pune', 'Ajay', 1),

(102, 'Meena', 'Mumbai', 'Vikas', 2),

(103, 'Kiran', 'Delhi', 'Sonia', 3),

(104, 'Arjun', 'Pune', 'Ajay', 4),

(105, 'Sita', 'Nashik', 'Vikas', 1),

(106, 'Rohit', 'Pune', 'Sonia', 5);

B. Queries (any 3)

i. List details of all customers who live in Pune

SELECT *

FROM Customer

WHERE city = 'Pune';

ii. Display the average premium amount

SELECT AVG(premium_amt) AS avg_premium

FROM Policy;

iii. Count customers who have taken 'Jeevan Anand' policy

SELECT COUNT(*) AS customer_count

FROM Customer

WHERE pno = (

SELECT pno

FROM Policy

WHERE pname = 'Jeevan Anand'

);

iv. Increase premium amount for 'Monthly' policies by 10%

UPDATE Policy

SET premium_amt = premium_amt * 1.10

WHERE policy_type = 'Monthly';

v. Policy_type wise count of policies

SELECT policy_type, COUNT(*) AS total_policies

FROM Policy

GROUP BY policy_type;

C. Choice

1. Policy_type with highest average premium amount

SELECT policy_type

FROM Policy

GROUP BY policy_type

ORDER BY AVG(premium_amt) DESC

LIMIT 1;

2. View: customer with highest number of policies

CREATE VIEW MaxPoliciesCustomer AS

SELECT cname, COUNT(*) AS total_policies

FROM Customer

GROUP BY cname

ORDER BY total_policies DESC

LIMIT 1;

Slip 5

Q.2) Consider the following database (primary keys are underlined).

Doctor (dno, dname, city)

Patient (pno, pat_name, city, disease)

The relationship is as follows: Doctor-Patient: many-to-many with Fees as

a

descriptive attribute.

A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostGreSQL(any 3)

i. List the details of all the doctors from 'Nagpur' city.

ii. Display count of patients.

iii. List the names of the patients suffering from 'cancer' disease.

iv. Change the city of 'Dr. Patil' to Pune.

v. List the names of patients that starts with alphabet 'A'

C. Write a query to find the number of patients suffering from "Asthama"

and have been charged a fee of Rs. 200 by "Dr. Kumar".

[4 marks]

OR

**C. Create a view to find the name of a doctor who treats maximum
number
of patients.**

-- Doctor table

```
CREATE TABLE Doctor (  
dno INT PRIMARY KEY,  
dname VARCHAR(50),  
city VARCHAR(50)  
);
```

-- Patient table

```
CREATE TABLE Patient (  
pno INT PRIMARY KEY,  
pat_name VARCHAR(50),  
city VARCHAR(50),  
disease VARCHAR(50)
```

);

-- Doctor-Patient relationship (many-to-many) with Fees

CREATE TABLE Doctor_Patient (

dno INT,

pno INT,

fees NUMERIC(10,2),

PRIMARY KEY (dno, pno),

FOREIGN KEY (dno) REFERENCES Doctor(dno),

FOREIGN KEY (pno) REFERENCES Patient(pno)

);

-- Insert sample doctors

INSERT INTO Doctor VALUES

(1, 'Dr. Patil', 'Nagpur'),

(2, 'Dr. Kumar', 'Delhi'),

(3, 'Dr. Mehta', 'Mumbai');

-- Insert sample patients

INSERT INTO Patient VALUES

```
(101, 'Arjun', 'Nagpur', 'Asthama'),  
(102, 'Asha', 'Pune', 'Cancer'),  
(103, 'Ravi', 'Delhi', 'Fever'),  
(104, 'Anita', 'Mumbai', 'Asthama'),  
(105, 'Meena', 'Nagpur', 'Cancer');
```

-- Insert sample doctor-patient relations with fees

```
INSERT INTO Doctor_Patient VALUES
```

```
(1, 101, 300),  
(1, 102, 500),  
(2, 101, 200),  
(2, 103, 250),  
(2, 104, 200),  
(3, 105, 400);
```

B. Queries (any 3)

i. Details of doctors from 'Nagpur'

```
SELECT *
```

FROM Doctor

WHERE city = 'Nagpur';

ii. Count of patients

SELECT COUNT(*) AS total_patients

FROM Patient;

iii. Names of patients suffering from 'Cancer'

SELECT pat_name

FROM Patient

WHERE disease = 'Cancer';

iv. Change city of 'Dr. Patil' to Pune

UPDATE Doctor

SET city = 'Pune'

WHERE dname = 'Dr. Patil';

v. Names of patients starting with 'A'

```
SELECT pat_name  
  
FROM Patient  
  
WHERE pat_name LIKE 'A%';
```

C. Choice

1. Patients with “Asthama” and fee = 200 by “Dr. Kumar”

```
SELECT COUNT(*) AS total_patients  
FROM Patient P  
JOIN Doctor_Patient DP ON P.pno = DP.pno  
JOIN Doctor D ON DP.dno = D.dno  
WHERE P.disease = 'Asthama'  
AND DP.fees = 200  
AND D.dname = 'Dr. Kumar'; without join
```

```
OR  
SELECT COUNT(*) AS total_patients  
FROM Patient  
WHERE disease = 'Asthama'  
AND pno IN (  
    SELECT pno  
    FROM Doctor_Patient  
    WHERE fees = 200  
    AND dno = (  
        SELECT dno  
        FROM Doctor  
        WHERE dname = 'Dr. Kumar'
```

);
)

2. View: Doctor who treats maximum number of patients

CREATE VIEW MaxPatientsDoctor AS

SELECT D.dname, COUNT(DP.pno) AS total_patients

FROM Doctor D

JOIN Doctor_Patient DP ON D.dno = DP.dno

GROUP BY D.dname

ORDER BY total_patients DESC

LIMIT 1;

Or

CREATE VIEW MaxPatientsDoctor AS

SELECT dname,

(SELECT COUNT(*)

FROM Doctor_Patient DP

WHERE DP.dno = D.dno) AS total_patients

FROM Doctor D

ORDER BY total_patients DESC

LIMIT 1;

Slip 6

Q 2) Consider the following database(primary keys are underlined).

Student (rno,name,city)

Teacher(tno,tname,phone_no,salary)

The relationship is as follows: Student-Teacher: many-to-many with subject as a descriptive attribute.

- A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]**
- B. Execute the following queries in PostgreSQL (any 3) [6 marks]**
- i. List the names of students from 'Ahilyanagar' city.**
 - ii. Display the count of teachers whose salary is between 40000 to 60000.**
 - iii. Change the phone number of 'Prof. Patil' to '9822131226'**
 - iv. List the details of the teachers in the sorted order of their name.**
 - v. List the names of the teachers who have salary less than 50000.**
- C. Write a query to list the names of all teachers who do not teach any subject that is taught by 'Mr. Patil'. [4 marks]**

OR

- C. Create a view to list the details of students who have taken 'DBMS' subject.**

-- Student table

CREATE TABLE Student (

rno INT PRIMARY KEY,


```
name VARCHAR(50),  
  
city VARCHAR(50)  
  
);
```

-- Teacher table

```
CREATE TABLE Teacher (  
  
    tno INT PRIMARY KEY,  
  
    tname VARCHAR(50),  
  
    phone_no VARCHAR(15),  
  
    salary NUMERIC(10,2)  
  
);
```

-- Relationship table with subject

```
CREATE TABLE Student_Teacher (  
  
    rno INT,  
  
    tno INT,  
  
    subject VARCHAR(50),  
  
    PRIMARY KEY (rno, tno, subject),  
  
    FOREIGN KEY (rno) REFERENCES Student(rno),  
  
    FOREIGN KEY (tno) REFERENCES Teacher(tno)
```

);

-- Insert Students

INSERT INTO Student VALUES

(1, 'Ravi', 'Pune'),

(2, 'Meena', 'Ahilyanagar'),

(3, 'Arjun', 'Mumbai'),

(4, 'Sita', 'Ahilyanagar');

-- Insert Teachers

INSERT INTO Teacher VALUES

(101, 'Prof. Patil', '9876543210', 45000),

(102, 'Mr. Kumar', '9898989898', 55000),

(103, 'Dr. Mehta', '9123456789', 65000),

(104, 'Mr. Joshi', '9234567890', 40000);

-- Insert Student-Teacher relations with subjects

INSERT INTO Student_Teacher VALUES

(1, 101, 'DBMS'),

(2, 101, 'DBMS'),

(3, 102, 'Maths'),
(4, 104, 'English'),
(2, 103, 'Science'),
(1, 104, 'English');

B. Queries (any 3)

i. List the names of students from 'Ahilyanagar'

```
SELECT name  
  
FROM Student  
  
WHERE city = 'Ahilyanagar';
```

ii. Count of teachers whose salary is between 40000 to 60000

```
SELECT COUNT(*) AS teacher_count  
  
FROM Teacher  
  
WHERE salary BETWEEN 40000 AND 60000;
```

iii. Change phone number of 'Prof. Patil'

UPDATE Teacher

SET phone_no = '9822131226'

WHERE tname = 'Prof. Patil';

iv. List teachers sorted by name

SELECT *

FROM Teacher

ORDER BY tname;

v. Names of teachers with salary < 50000

SELECT tname

FROM Teacher

WHERE salary < 50000;

C. Choice

1. Teachers who do not teach any subject taught by 'Mr. Patil'

```
SELECT tname
FROM Teacher
WHERE tno NOT IN (
    SELECT tno
    FROM Student_Teacher
    WHERE subject IN (
        SELECT subject
        FROM Student_Teacher
        WHERE tno = (
            SELECT tno FROM Teacher WHERE tname = 'Mr. Patil'
        )
    )
);
```

2. Create a view: students who have taken 'DBMS' subject

```
CREATE VIEW DBMS_Students AS
SELECT S.rno, S.name, S.city
FROM Student S
```

```
WHERE S.rno IN (  
  
    SELECT rno  
  
    FROM Student_Teacher  
  
    WHERE subject = 'DBMS'  
  
);
```

Slip 7

Q 2) Consider the following database(primary keys are underlined).

Item (itemno, name, quantity)

Supplier (sno, name, city)

**The relationship is as follows: Item-Supplier: many-to-many with rate as a
descriptive attribute.**

**A. Create the above database in PostGreSQL and insert sufficient
records. [5 marks]**

B. Execute the following queries in PostGreSQL (any 3) [6 marks]

- i. Change the quantity of item 'Mouse' to 80.**
- ii. List the details of the suppliers whose name begins with the alphabet
'M'.**
- iii. Display the total count of items.**
- iv. List the names of suppliers who do not live in Pune city.**
- v. List the names of items with quantity less than 10.**

C. Write a query to display the name of supplier who supplies Item with minimum rate.

[4 marks]

OR

C. Create a view to display supplier-wise list of items.

-- Item table

```
CREATE TABLE Item (  
    itemno INT PRIMARY KEY,  
    name VARCHAR(50),  
    quantity INT  
);
```

-- Supplier table

```
CREATE TABLE Supplier (  
    sno INT PRIMARY KEY,  
    name VARCHAR(50),  
    city VARCHAR(50)  
);
```

-- Relationship table with descriptive attribute (rate)

```
CREATE TABLE Item_Supplier (  
  
    itemno INT,  
  
    sno INT,  
  
    rate NUMERIC(10,2),  
  
    PRIMARY KEY (itemno, sno),  
  
    FOREIGN KEY (itemno) REFERENCES Item(itemno),  
  
    FOREIGN KEY (sno) REFERENCES Supplier(sno)  
  
);
```

-- Insert Items

```
INSERT INTO Item VALUES  
  
(1, 'Keyboard', 50),  
  
(2, 'Mouse', 40),  
  
(3, 'Monitor', 15),  
  
(4, 'USB Cable', 5);
```

-- Insert Suppliers

```
INSERT INTO Supplier VALUES  
  
(101, 'Mahesh Traders', 'Pune'),
```


(102, 'Meera Electronics', 'Mumbai'),

(103, 'Raj Suppliers', 'Nashik'),

(104, 'Mohan Distributors', 'Nagpur');

-- Insert Item-Supplier relationships with rate

INSERT INTO Item_Supplier VALUES

(1, 101, 500),

(2, 101, 250),

(3, 102, 7000),

(4, 103, 100),

(2, 104, 230),

(3, 104, 6800);

B. Queries (any 3)

i. Change quantity of item 'Mouse' to 80

UPDATE Item

SET quantity = 80

WHERE name = 'Mouse';

ii. Suppliers whose name begins with 'M'

```
SELECT *  
  
FROM Supplier  
  
WHERE name LIKE 'M%';
```

iii. Total count of items

```
SELECT COUNT(*) AS total_items  
  
FROM Item;
```

iv. Suppliers not living in Pune

```
SELECT name  
  
FROM Supplier  
  
WHERE city <> 'Pune';
```

v. Items with quantity < 10

```
SELECT name
```

FROM Item

WHERE quantity < 10;

C. Choice

1. Supplier who supplies Item with minimum rate

SELECT S.name

FROM Supplier S

WHERE S.sno = (

 SELECT sno

 FROM Item_Supplier

 WHERE rate = (SELECT MIN(rate) FROM Item_Supplier)

);

2. View: supplier-wise list of items

CREATE VIEW Supplier_Item_List AS

SELECT S.name AS supplier_name, I.name AS item_name, IS1.rate

FROM Supplier S, Item I, Item_Supplier IS1

WHERE S.sno = IS1.sno AND I.itemno = IS1.itemno;

Slip 8

Q 2) Consider the following Student-Teacher database(primary keys are underlined).

Student (sno , s_name, s_class)

Teacher (tno , t_name, yrs_experience)

The relationship is as follows:

Student-Teacher: Many to Many with descriptive attribute Subject.

Constraint:s_class should be either "FY", "SY" or "TY"

A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostGreSQL (any 3) [6 marks]

i. Give class-wise number of students.

ii. List all students studying in class "TY".

iii. Count the number of students who have taken subject "Maths"

iv. Delete record of student whose sno "105"

v. Add designation column in teacher table

C. Write a query to list the names of all teachers with their subjects along with the total number of students they are teaching. [4 marks]

OR

C. Create a view to find the teacher who teaches maximum number of subjects.

-- Student table

CREATE TABLE Student (

sno INT PRIMARY KEY,

s_name VARCHAR(50),

s_class VARCHAR(5) CHECK (s_class IN ('FY','SY','TY'))

);

-- Teacher table

```
CREATE TABLE Teacher (  
  
    tno INT PRIMARY KEY,  
  
    t_name VARCHAR(50),  
  
    yrs_experience INT  
  
);
```

-- Relationship table with descriptive attribute (Subject)

```
CREATE TABLE Student_Teacher (  
  
    sno INT,  
  
    tno INT,  
  
    subject VARCHAR(50),  
  
    PRIMARY KEY (sno, tno, subject),  
  
    FOREIGN KEY (sno) REFERENCES Student(sno),  
  
    FOREIGN KEY (tno) REFERENCES Teacher(tno)  
  
);
```

-- Insert Students

```
INSERT INTO Student VALUES  
  
(101, 'Ravi', 'FY'),
```

(102, 'Meena', 'SY'),

(103, 'Amit', 'TY'),

(104, 'Sonal', 'TY'),

(105, 'Kiran', 'SY');

-- Insert Teachers

INSERT INTO Teacher VALUES

(201, 'Prof. Patil', 10),

(202, 'Prof. Mehta', 7),

(203, 'Prof. Sharma', 12);

-- Insert Student-Teacher with subjects

INSERT INTO Student_Teacher VALUES

(101, 201, 'DBMS'),

(102, 201, 'Maths'),

(103, 202, 'Maths'),

(104, 203, 'OS'),

(105, 202, 'Maths'),

(103, 201, 'DBMS');

B. Queries (any 3)

i. Class-wise number of students

```
SELECT s_class, COUNT(*) AS total_students  
  
FROM Student  
  
GROUP BY s_class;
```

ii. All students in TY

```
SELECT *  
  
FROM Student  
  
WHERE s_class = 'TY';
```

iii. Count students who have taken Maths

```
SELECT COUNT(DISTINCT sno) AS maths_students  
  
FROM Student_Teacher  
  
WHERE subject = 'Maths';
```

iv. Delete record of student with sno = 105

DELETE FROM Student

WHERE sno = 105;

v. Add designation column in teacher table

ALTER TABLE Teacher

ADD COLUMN designation VARCHAR(50);

C. Choice

1. Teachers with subjects & total number of students

SELECT T.t_name, ST.subject, COUNT(DISTINCT ST.sno) AS

total_students

FROM Teacher T, Student_Teacher ST

WHERE T.tno = ST.tno

GROUP BY T.t_name, ST.subject;

2. View: Teacher who teaches maximum number of subjects

```
CREATE VIEW MaxSubjectsTeacher AS

SELECT tno, COUNT(DISTINCT subject) AS subject_count

FROM Student_Teacher

GROUP BY tno

ORDER BY subject_count DESC

LIMIT 1;
```

Slip 9

Q 2) Create the following tables (primary keys are underlined).

Person (pnumber, pname, birthdate, income),

Area(aname,area_type).

There exists a one-to-many relationship between Area and Person.

Constraint: 'area_type' should have value either urban or rural.

A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostGreSQL (any 3) [6 marks]

i. List the details of persons with income greater than 10000.

ii. Transfer all people living in 'Pune' to 'Delhi'.

iii. Count number of area's which has type 'urban'.

iv. Give the count of people who are born in the month of February.

v. List the names of all people living in 'rural' area.

C. Write a query that count the total number of people staying in "urban" area with income equal to average income. [4 marks]

OR

C. Create a view to print the person's details along with their area name.

-- Area table

```
CREATE TABLE Area (
    aname VARCHAR(50) PRIMARY KEY,
    area_type VARCHAR(10) CHECK (area_type IN ('urban','rural'))
);
```

```
-- Person table (each person belongs to an area)
CREATE TABLE Person (
  pnumber INT PRIMARY KEY,
  pname VARCHAR(50),
  birthdate DATE,
  income NUMERIC(10,2),
  aname VARCHAR(50),
  FOREIGN KEY (aname) REFERENCES Area(aname)
);
```

```
-- Insert into Area
INSERT INTO Area VALUES
('Pune', 'urban'),
('Delhi', 'urban'),
('Nashik', 'rural'),
('Camp', 'urban');
```

```
-- Insert into Person
INSERT INTO Person VALUES
(101, 'Ravi', '1995-02-15', 12000, 'Pune'),
(102, 'Meena', '1997-07-21', 8000, 'Nashik'),
(103, 'Amit', '2000-02-05', 10000, 'Delhi'),
(104, 'Sonal', '1999-12-11', 15000, 'Pune'),
(105, 'Kiran', '2001-06-25', 12000, 'Camp');
```

B. Queries (any 3)

i. Persons with income > 10000

```
SELECT *
FROM Person
WHERE income > 10000;
```

ii. Transfer all people from Pune → Delhi

```
UPDATE Person
SET aname = 'Delhi'
WHERE aname = 'Pune';
```

iii. Count areas of type 'urban'

```
SELECT COUNT(*) AS total_urban
```

```
FROM Area
WHERE area_type = 'urban';
```

iv. Count people born in February

```
SELECT COUNT(*) AS feb_birthdays
FROM Person
WHERE EXTRACT(MONTH FROM birthdate) = 2;
```

v. List names of people living in rural areas

```
SELECT pname
FROM Person
WHERE aname IN (
    SELECT aname FROM Area WHERE area_type = 'rural'
);
```

C. Choice

1. Count people in urban areas with income equal to average income

```
SELECT COUNT(*) AS total_people
FROM Person
WHERE income = (SELECT AVG(income) FROM Person)
AND aname IN (SELECT aname FROM Area WHERE area_type = 'urban');
```

2. View: Person details with area name

```
CREATE VIEW PersonAreaView AS
SELECT P.pnumber, P.pname, P.birthdate, P.income, A.aname,
A.area_type
FROM Person P, Area A
WHERE P.aname = A.aname;
```

Slip 10

Q 2) Create the following tables (primary keys are underlined).

Account (acctno ,acct_type, balance, branch_name)

Customer (custno, cust_name, cust_city)

Relationships: Customer-Account: One to Many.

Constraints: acct_type should be “saving” or “current” and balance is greater than zero.

A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostgreSQL (any 3) [6 marks]

i. Display information of all saving accounts having balance greater than 5000.

ii. Count customers in city “Pune”.

iii. Find the total balance at branch “M.G.Road”.

iv. Delete the record whose cust_name is “Ramnath”.

v. Change city of customer “Sanjay” to “Pune”

C. Write a query to find the number of account holders for each city. [4 marks]

OR

C. Create a view to print the account details along with customer and their branch details.

-- Customer table

```
CREATE TABLE Customer (  
    custno INT PRIMARY KEY,  
    cust_name VARCHAR(50),  
    cust_city VARCHAR(50)  
);
```

-- Account table

```
CREATE TABLE Account (  
    acctno INT PRIMARY KEY,  
    acct_type VARCHAR(10) CHECK (acct_type IN ('saving','current')),  
    balance NUMERIC(12,2) CHECK (balance > 0),  
    branch_name VARCHAR(50),  
    custno INT,  
    FOREIGN KEY (custno) REFERENCES Customer(custno)  
);
```

-- Insert Customers

```
INSERT INTO Customer VALUES  
(1, 'Ravi', 'Pune'),  
(2, 'Meena', 'Mumbai'),  
(3, 'Sanjay', 'Delhi'),  
(4, 'Ramnath', 'Nashik');
```

-- Insert Accounts

```
INSERT INTO Account VALUES  
(101, 'saving', 7000, 'M.G.Road', 1),  
(102, 'current', 12000, 'Deccan', 2),  
(103, 'saving', 3000, 'M.G.Road', 3),  
(104, 'saving', 15000, 'Camp', 1),  
(105, 'current', 9000, 'M.G.Road', 4);
```

B. Queries (any 3)

i. All saving accounts with balance > 5000

```
SELECT *  
FROM Account  
WHERE acct_type = 'saving'  
AND balance > 5000;
```

ii. Count customers in Pune

```
SELECT COUNT(*) AS pune_customers  
FROM Customer  
WHERE cust_city = 'Pune';
```

iii. Total balance at branch M.G.Road

```
SELECT SUM(balance) AS total_balance  
FROM Account  
WHERE branch_name = 'M.G.Road';
```

iv. Delete customer named Ramnath

```
DELETE FROM Customer  
WHERE cust_name = 'Ramnath';
```

v. Change city of customer Sanjay → Pune

```
UPDATE Customer  
SET cust_city = 'Pune'  
WHERE cust_name = 'Sanjay';
```

C. any one

1. Number of account holders per city

```
SELECT C.cust_city, COUNT(A.acctno) AS total_accounts
FROM Customer C, Account A
WHERE C.custno = A.custno
GROUP BY C.cust_city;
```

2. View: Account details with customer + branch

```
CREATE VIEW AccountCustomerView AS
SELECT A.acctno, A.acct_type, A.balance, A.branch_name,
       C.cust_name, C.cust_city
FROM Account A, Customer C
WHERE A.custno = C.custno;
```

Slip 11

Q 2) Create the following tables (primary keys are underlined).

Emp (eno, ename, salary)

Project (pno, pname, budget)

Relationships:

Emp– Project : Many to Many with descriptive attribute total_hours.

A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostgreSQL (any 3) [6 marks]

i. Find the maximum budget.

ii. Increase the salary of all employees by 10%.

iii. Count the number of projects having duration greater than 100 hrs.

iv. List all employees whose name ends with “a”.

v. Add column contact_number to employee table.

**C. Write a query to list the names of employees who are not working on any project.
[4 marks]**

OR

C. Create a view to list the project details having maximum number of hours.

-- Employee table

```
CREATE TABLE Emp (
  eno INT PRIMARY KEY,
```

```

    ename VARCHAR(50),
    salary NUMERIC(10,2)
);

-- Project table
CREATE TABLE Project (
    pno INT PRIMARY KEY,
    pname VARCHAR(50),
    budget NUMERIC(12,2)
);

-- Relationship table with descriptive attribute (total_hours)
CREATE TABLE Emp_Project (
    eno INT,
    pno INT,
    total_hours INT,
    PRIMARY KEY (eno, pno),
    FOREIGN KEY (eno) REFERENCES Emp(eno),
    FOREIGN KEY (pno) REFERENCES Project(pno)
);

-- Insert Employees
INSERT INTO Emp VALUES
(1, 'Ravi', 25000),
(2, 'Meena', 30000),
(3, 'Sonia', 28000),
(4, 'Amit', 35000);

-- Insert Projects
INSERT INTO Project VALUES
(101, 'DBMS', 500000),
(102, 'AI', 800000),
(103, 'Networking', 400000);

-- Insert Emp_Project (who works where & hours)
INSERT INTO Emp_Project VALUES
(1, 101, 120),
(2, 101, 80),
(2, 102, 150),
(3, 103, 90);

```

B. Queries (any 3)

i. Find the maximum budget

```
SELECT MAX(budget) AS max_budget  
FROM Project;
```

ii. Increase salary of all employees by 10%

```
UPDATE Emp  
SET salary = salary * 1.10;
```

iii. Count projects with duration > 100 hrs

```
SELECT COUNT(DISTINCT pno) AS project_count  
FROM Emp_Project  
WHERE total_hours > 100;
```

iv. Employees whose name ends with 'a'

```
SELECT *  
FROM Emp  
WHERE ename LIKE '%a';
```

v. Add column contact_number

```
ALTER TABLE Emp  
ADD COLUMN contact_number VARCHAR(15);
```

C. Choice

1. Employees not working on any project

```
SELECT ename  
FROM Emp  
WHERE eno NOT IN (SELECT eno FROM Emp_Project);
```

2. View: Project with maximum hours

```
CREATE VIEW MaxHoursProject AS  
SELECT P.pno, P.pname, P.budget, SUM(EP.total_hours) AS total_hours  
FROM Project P, Emp_Project EP  
WHERE P.pno = EP.pno  
GROUP BY P.pno, P.pname, P.budget
```


ORDER BY total_hours DESC
LIMIT 1;

Slip 12

Q 2) Create the following tables (primary keys are underlined).

Sales_order(sorderno, s_order_date, order_amt)

Client(clientno, name, address)

There exists a one-to-many relationship between Client and Sales_order

Constraints: order_amt should be greater than zero.

A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostgreSQL (any 3) [6 marks]

i. Update the client address of all clients from "Nagpur" to "Kolhapur".

ii. Delete all sales order whose client number is 30;

iii. Display all sale records having order date before "23/02/2024".

iv. Display sales order having maximum order amount.

v. Add column order_status to the Sales_order table.

C. Write a query to Display client details having maximum sales orders. [4 marks]

OR

C. Create a view to list names of clients having more than 2 sales orders.

-- Client table

```
CREATE TABLE Client (  
  clientno INT PRIMARY KEY,  
  name VARCHAR(50),  
  address VARCHAR(100)  
);
```

-- Sales_order table

```
CREATE TABLE Sales_order (  
  sorderno INT PRIMARY KEY,  
  s_order_date DATE,  
  order_amt NUMERIC(12,2) CHECK (order_amt > 0),  
  clientno INT,  
  FOREIGN KEY (clientno) REFERENCES Client(clientno)  
);
```

-- Insert Clients

```
INSERT INTO Client VALUES  
(10, 'Ravi', 'Nagpur'),  
(20, 'Meena', 'Pune'),  
(30, 'Amit', 'Mumbai'),  
(40, 'Sonal', 'Nagpur');
```

-- Insert Sales Orders

```
INSERT INTO Sales_order VALUES  
(101, '2024-02-20', 12000, 10),  
(102, '2024-02-25', 15000, 20),  
(103, '2024-01-15', 8000, 30),  
(104, '2024-02-22', 25000, 10),  
(105, '2024-03-01', 20000, 40);
```

B. Queries (any 3)

i. Update Nagpur → Kolhapur

```
UPDATE Client  
SET address = 'Kolhapur'  
WHERE address = 'Nagpur';
```

ii. Delete all sales orders for clientno = 30

```
DELETE FROM Sales_order  
WHERE clientno = 30;
```

iii. Sales records before 23/02/2024

```
SELECT *  
FROM Sales_order  
WHERE s_order_date < '2024-02-23';
```

iv. Sales order with maximum amount

```
SELECT *  
FROM Sales_order  
WHERE order_amt = (SELECT MAX(order_amt) FROM Sales_order);
```

v. Add column order_status

```
ALTER TABLE Sales_order  
ADD COLUMN order_status VARCHAR(20);
```

C. Choice

1. Client with maximum sales orders

```
SELECT C.clientno, C.name, C.address, COUNT(S.sorderno) AS  
total_orders  
FROM Client C, Sales_order S  
WHERE C.clientno = S.clientno  
GROUP BY C.clientno, C.name, C.address  
ORDER BY total_orders DESC  
LIMIT 1;
```

2. View: Clients having more than 2 sales orders

```
CREATE VIEW ClientsWithMoreThan2Orders AS  
SELECT C.clientno, C.name, COUNT(S.sorderno) AS total_orders  
FROM Client C, Sales_order S  
WHERE C.clientno = S.clientno  
GROUP BY C.clientno, C.name  
HAVING COUNT(S.sorderno) > 2;
```

Slip 13

Q 2) Create the following tables (primary keys are underlined).

Bus (Busno , capacity ,depot_name)

Route (Routeno ,source ,destination ,no_of_stations)

Relationship: Bus-Route : Many to one

Constraint: Bus capacity should not be null.

A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostGreSQL (any 3) [6 marks]

i. List all buses at depot "Swargate".

ii. Delete Bus whose Bus number is "204".

iii. List all buses on route number 41.

iv. List the route details having number of stations greater than 10.

v. List all routes starting from "Pune station".

C. Write a query to delete all the buses on routes in which total stations are less than 3.

[4 marks]

OR

C. Create a view to list all buses and route details from "Pimpri" to

“Hadapsar”.

-- Create Route table

```
CREATE TABLE Route (  
    Routeno INT PRIMARY KEY,  
    source VARCHAR(50),  
    destination VARCHAR(50),  
    no_of_stations INT  
);
```

-- Create Bus table

```
CREATE TABLE Bus (  
    Busno INT PRIMARY KEY,  
    capacity INT NOT NULL, -- Constraint: cannot be null  
    depot_name VARCHAR(50),  
    Routeno INT, -- Foreign key (Many buses → One route)  
    FOREIGN KEY (Routeno) REFERENCES Route(Routeno)  
);
```

-- Insert some sample records into Route

```
INSERT INTO Route VALUES (41, 'Pune Station', 'Kothrud', 12);  
INSERT INTO Route VALUES (42, 'Swargate', 'Nigdi', 15);  
INSERT INTO Route VALUES (43, 'Pimpri', 'Hadapsar', 18);  
INSERT INTO Route VALUES (44, 'Katraj', 'Shivaji Nagar', 2);
```

-- Insert some sample records into Bus

```
INSERT INTO Bus VALUES (201, 50, 'Swargate', 41);  
INSERT INTO Bus VALUES (202, 45, 'Nigdi', 42);  
INSERT INTO Bus VALUES (203, 60, 'Swargate', 43);  
INSERT INTO Bus VALUES (204, 40, 'Kothrud', 42);  
INSERT INTO Bus VALUES (205, 55, 'Pune Station', 44);
```

B. Queries (Any 3)

i) List all buses at depot “Swargate”

```
SELECT * FROM Bus WHERE depot_name = 'Swargate';
```

ii) Delete Bus whose Bus number is “204”

```
DELETE FROM Bus WHERE Busno = 204;
```

iii) List all buses on route number 41

```
SELECT * FROM Bus WHERE Routeno = 41;
```

iv) List the route details having number of stations greater than 10

```
SELECT * FROM Route WHERE no_of_stations > 10;
```

v) List all routes starting from “Pune station”

```
SELECT * FROM Route WHERE source = 'Pune Station';
```

C. Two Options

Option 1: Delete buses on routes with < 3 stations

```
DELETE FROM Bus
WHERE Routeno IN (
    SELECT Routeno FROM Route WHERE no_of_stations < 3
);
```

OR

Option 2: Create a view (Pimpri → Hadapsar)

```
CREATE VIEW Pimpri_Hadapsar_Buses AS
SELECT b.Busno, b.capacity, b.depot_name, r.Routeno, r.source,
r.destination, r.no_of_stations
FROM Bus b
JOIN Route r ON b.Routeno = r.Routeno
WHERE r.source = 'Pimpri' AND r.destination = 'Hadapsar';
```

Slip 14

Q 2) Consider the following database (Primary keys are underlined)
Doctor (dcode, doctor_name, specialization, address, phone_no, fees)

Patient (pcode, Patient_name, symptoms)

There exists a one-to-many relationship between Doctor and Patient.

A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostgreSQL (any 3) [6 marks]

i. Find the names of all Patients which start with “M”.

ii. Count the number of doctors who are Neurologists.

- iii. Give the list of all patients who are suffering from “Fever”
 - iv. Find the specialization and phone numbers of all doctors from ‘Sadashiv Peth’.
 - v. Change the address of Dr. Patil to “Camp”.
- C. Write a query to display the list of all patients who are treated by doctors staying in ‘Shivajinagar’ area with the specialization “Orthopedic”. [4 marks]

OR

C. Create a view to find the names of patients who are treated by doctor with minimum fees.

```
-- Create Doctor table
CREATE TABLE Doctor (
  dcode INT PRIMARY KEY,
  doctor_name VARCHAR(50),
  specialization VARCHAR(50),
  address VARCHAR(100),
  phone_no VARCHAR(15),
  fees INT
);

-- Create Patient table
CREATE TABLE Patient (
  pcode INT PRIMARY KEY,
  patient_name VARCHAR(50),
  symptoms VARCHAR(50),
  dcode INT, -- foreign key (many patients → one doctor)
  FOREIGN KEY (dcode) REFERENCES Doctor(dcode)
);

-- Insert sample doctors
INSERT INTO Doctor VALUES (1, 'Dr. Patil', 'Cardiologist', 'Sadashiv Peth', '9876543210', 500);
INSERT INTO Doctor VALUES (2, 'Dr. Joshi', 'Neurologist', 'Shivajinagar', '9898989898', 700);
INSERT INTO Doctor VALUES (3, 'Dr. Kulkarni', 'Orthopedic', 'Shivajinagar', '9765432100', 600);
INSERT INTO Doctor VALUES (4, 'Dr. Deshmukh', 'Neurologist', 'Camp', '9123456789', 800);

-- Insert sample patients
INSERT INTO Patient VALUES (101, 'Meera', 'Fever', 1);
INSERT INTO Patient VALUES (102, 'Mohan', 'Headache', 2);
```

```
INSERT INTO Patient VALUES (103, 'Sita', 'Fever', 3);
INSERT INTO Patient VALUES (104, 'Manish', 'Cough', 2);
INSERT INTO Patient VALUES (105, 'Anita', 'Cold', 4);
```

B. Queries (Any 3)

i) Find the names of all Patients which start with “M”

```
SELECT patient_name FROM Patient
WHERE patient_name LIKE 'M%';
```

ii) Count the number of doctors who are Neurologists

```
SELECT COUNT(*) AS neurologist_count
FROM Doctor
WHERE specialization = 'Neurologist';
```

iii) Give the list of all patients who are suffering from “Fever”

```
SELECT patient_name FROM Patient
WHERE symptoms = 'Fever';
```

iv) Find the specialization and phone numbers of all doctors from ‘Sadashiv Peth’

```
SELECT specialization, phone_no FROM Doctor
WHERE address = 'Sadashiv Peth';
```

v) Change the address of Dr. Patil to “Camp”

```
UPDATE Doctor
SET address = 'Camp'
WHERE doctor_name = 'Dr. Patil';
```

C. Two Options

Option 1: Patients treated by Orthopedic doctors in Shivajinagar

```
SELECT p.patient_name
FROM Patient p
JOIN Doctor d ON p.dcode = d.dcode
```

WHERE d.address = 'Shivajinagar'
AND d.specialization = 'Orthopedic';

Option 2: Create a view for patients treated by doctor with minimum fees

```
CREATE VIEW MinFeesPatients AS
SELECT p.patient_name, d.doctor_name, d.fees
FROM Patient p
JOIN Doctor d ON p.dcode = d.dcode
WHERE d.fees = (SELECT MIN(fees) FROM Doctor);
```

Slip 15

Q 2) Create the following tables (primary keys are underlined).

Game (gname, noofplayers, coachname, captain_name)

Player (pno, pname)

There exists a one-to-many relationship between Game and Player.

Constraint captain_name should be uppercase.

A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostgreSQL (any 3) [6 marks]

i. List the names of players playing “hockey”.

ii. Give the average number of players.

iii. Delete the records of players playing ‘kho kho’.

iv. List names of players not playing “cricket”.

v. Update the coach name from “Dhoni” to “Tendulkar” for game “Cricket”.

C. Write a query to display the game details with maximum number of players. [4 marks]

OR

C. Create a view to print game-wise list of Players along with their game name

-- Create Game table


```
CREATE TABLE Game (  
    gname VARCHAR(30) PRIMARY KEY,  
    noofplayers INT,  
    coachname VARCHAR(30),  
    captain_name VARCHAR(30) CHECK (captain_name =  
UPPER(captain_name)) -- must be uppercase  
);
```

-- Create Player table

```
CREATE TABLE Player (  
    pno INT PRIMARY KEY,  
    pname VARCHAR(30),  
    gname VARCHAR(30),  
    FOREIGN KEY (gname) REFERENCES Game(gname)  
);
```

-- Insert sample games

```
INSERT INTO Game VALUES ('Cricket', 11, 'Dhoni', 'KOHLI');  
INSERT INTO Game VALUES ('Hockey', 11, 'Singh', 'RAHUL');  
INSERT INTO Game VALUES ('Kho Kho', 12, 'Sharma', 'AMAN');  
INSERT INTO Game VALUES ('Football', 11, 'Jose', 'ROHIT');
```

-- Insert sample players

```
INSERT INTO Player VALUES (1, 'Amit', 'Cricket');  
INSERT INTO Player VALUES (2, 'Ravi', 'Cricket');  
INSERT INTO Player VALUES (3, 'Sohan', 'Hockey');  
INSERT INTO Player VALUES (4, 'Mohan', 'Hockey');  
INSERT INTO Player VALUES (5, 'Raj', 'Kho Kho');  
INSERT INTO Player VALUES (6, 'Anil', 'Football');  
INSERT INTO Player VALUES (7, 'Sunil', 'Football');
```

B. Queries (Any 3)

i. List the names of players playing “hockey”

```
SELECT pname FROM Player  
WHERE gname = 'Hockey';
```

ii. Give the average number of players

```
SELECT AVG(noofplayers) AS avg_players
```

FROM Game;

iii. Delete the records of players playing 'kho kho'

```
DELETE FROM Player
WHERE gname = 'Kho Kho';
```

iv. List names of players not playing "cricket"

```
SELECT pname FROM Player
WHERE gname <> 'Cricket';
```

v. Update the coach name from "Dhoni" to "Tendulkar" for game "Cricket"

```
UPDATE Game
SET coachname = 'Tendulkar'
WHERE gname = 'Cricket' AND coachname = 'Dhoni';
```

C. Two Options

Option 1: Display game details with maximum number of players

```
SELECT * FROM Game
WHERE noofplayers = (SELECT MAX(noofplayers) FROM Game);
```

Option 2: Create a view to print game-wise list of players

```
CREATE VIEW GameWisePlayers AS
SELECT g.gname, p.pname
FROM Game g
JOIN Player p ON g.gname = p.gname;
```

Q 2) Consider the following database (primary keys are underlined).

Student (sno ,s_name, s_class)

Teacher (tno ,t_name, yrs_experience)

The relationship is as follows:

Student-Teacher: M-M with descriptive attribute Subject.

Constraint: s_class should be either "FY", "SY" or "TY"

A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostGreSQL (any 3) [6 marks]

i. List teachers who has more than 5 years of experience.

ii. Count all students who have taken subject "Data Structure"

iii. List the names of all students from SY Class.

iv. Change experience of teacher named "Pawar" to 20.

v. Display classwise details of students.

C. Write a query to list the names of all teachers with their subjects along with the student names to whom they are teaching. [4 marks]

OR

C. Create a view to find the teacher teaching maximum number of subjects

-- Create Student table

CREATE TABLE Student (

sno INT PRIMARY KEY,

s_name VARCHAR(50),

s_class VARCHAR(5) CHECK (s_class IN ('FY','SY','TY')) -- constraint

);

-- Create Teacher table

CREATE TABLE Teacher (

```
tno INT PRIMARY KEY,  
  
t_name VARCHAR(50),  
  
yrs_experience INT  
  
);
```

-- Create relationship table with Subject

```
CREATE TABLE StudentTeacher (  
  
sno INT,  
  
tno INT,  
  
subject VARCHAR(50),  
  
PRIMARY KEY (sno, tno, subject),  
  
FOREIGN KEY (sno) REFERENCES Student(sno),  
  
FOREIGN KEY (tno) REFERENCES Teacher(tno)  
  
);
```

-- Insert Students

```
INSERT INTO Student VALUES (1, 'Amit', 'FY');  
  
INSERT INTO Student VALUES (2, 'Sneha', 'SY');  
  
INSERT INTO Student VALUES (3, 'Ravi', 'SY');  
  
INSERT INTO Student VALUES (4, 'Meera', 'TY');
```

-- Insert Teachers

INSERT INTO Teacher VALUES (101, 'Patil', 10);

INSERT INTO Teacher VALUES (102, 'Pawar', 5);

INSERT INTO Teacher VALUES (103, 'Kulkarni', 8);

-- Insert Relationship (Student–Teacher–Subject)

INSERT INTO StudentTeacher VALUES (1, 101, 'C Programming');

INSERT INTO StudentTeacher VALUES (2, 101, 'Data Structure');

INSERT INTO StudentTeacher VALUES (3, 102, 'Data Structure');

INSERT INTO StudentTeacher VALUES (4, 103, 'DBMS');

INSERT INTO StudentTeacher VALUES (2, 103, 'Operating System');

B. Queries (Any 3)

i. List teachers who has more than 5 years of experience

SELECT t_name FROM Teacher

WHERE yrs_experience > 5;

ii. Count all students who have taken subject “Data Structure”

SELECT COUNT(DISTINCT sno) AS student_count

FROM StudentTeacher

WHERE subject = 'Data Structure';

iii. List the names of all students from SY Class

SELECT s_name FROM Student

WHERE s_class = 'SY';

iv. Change experience of teacher named “Pawar” to 20

UPDATE Teacher

SET yrs_experience = 20

WHERE t_name = 'Pawar';

v. Display classwise details of students

```
SELECT s_class, s_name
```

```
FROM Student
```

```
ORDER BY s_class;
```

C. Two Options

Option 1: List teachers with their subjects and student names

```
SELECT t.t_name, st.subject, s.s_name
```

```
FROM Teacher t
```

```
JOIN StudentTeacher st ON t.tno = st.tno
```

```
JOIN Student s ON st.sno = s.sno;
```

Option 2: Create a view to find the teacher teaching maximum number of subjects

```

CREATE VIEW MaxSubjectTeacher AS

SELECT t.t_name, COUNT(st.subject) AS subject_count

FROM Teacher t

JOIN StudentTeacher st ON t.tno = st.tno

GROUP BY t.t_name

ORDER BY subject_count DESC

LIMIT 1;

```

Slip17

Q 2) Consider the following database (Primary keys are underlined)

Property (Pno, area_Sqft, location, city)

Owner (oname, address, phone)

The relationship is as follows:

Property – Owner are related with Many to One relationship.

- A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]**
- B. Execute the following queries in PostGreSQL (any 3) [6 marks]**
 - i. Give the list of all properties in Nashik city.**
 - ii. Give the names of owners whose property is greater than 2000sq feet.**
 - iii. Find the number of properties owned by “Mr.Rahane”.**
 - iv. Count all properties of “Pune” city.**
 - v. Delete all properties in “Pune” city owned by “Mr.Baravkar”.**
- C. Write a query to find the names of all owners who own a property in Pune city, located in ‘Kothrud’ area and which has an area greater than 10000sqft.**

OR

- C. Create a view to find the owner wise list of properties.**

-- Create Owner table

```

CREATE TABLE Owner (

```



```
    oname VARCHAR(50) PRIMARY KEY,  
  
    address VARCHAR(100),  
  
    phone VARCHAR(15)  
  
);
```

-- Create Property table (Many properties → One owner)

```
CREATE TABLE Property (  
  
    pno INT PRIMARY KEY,  
  
    area_sqft INT,  
  
    location VARCHAR(50),  
  
    city VARCHAR(50),  
  
    oname VARCHAR(50),  
  
    FOREIGN KEY (oname) REFERENCES Owner(oname)  
  
);
```

-- Insert Owners

```
INSERT INTO Owner VALUES ('Mr.Rahane', 'Nashik', '9876543210');  
  
INSERT INTO Owner VALUES ('Mr.Baravkar', 'Pune', '9123456789');  
  
INSERT INTO Owner VALUES ('Mr.Kulkarni', 'Mumbai', '9988776655');
```

-- Insert Properties

```
INSERT INTO Property VALUES (1, 2500, 'Gangapur Road', 'Nashik',  
'Mr.Rahane');
```

```
INSERT INTO Property VALUES (2, 1800, 'Kothrud', 'Pune',  
'Mr.Baravkar');
```

```
INSERT INTO Property VALUES (3, 12000, 'Kothrud', 'Pune',  
'Mr.Kulkarni');
```

```
INSERT INTO Property VALUES (4, 3000, 'Deccan', 'Pune', 'Mr.Rahane');
```

```
INSERT INTO Property VALUES (5, 900, 'Dadar', 'Mumbai', 'Mr.Kulkarni');
```

B. Queries (Any 3)

i. List of all properties in Nashik city

```
SELECT * FROM Property  
WHERE city = 'Nashik';
```

ii. Owners whose property > 2000 sqft

```
SELECT oname
```

FROM Property

WHERE area_sqft > 2000;

iii. Number of properties owned by “Mr.Rahane”

SELECT COUNT(*) AS property_count

FROM Property

WHERE oname = 'Mr.Rahane';

iv. Count all properties of Pune city

SELECT COUNT(*) AS pune_properties

FROM Property

WHERE city = 'Pune';

v. Delete all properties in Pune owned by “Mr.Baravkar”

DELETE FROM Property

WHERE city = 'Pune' AND oname = 'Mr.Baravkar';

Option 1 (Simple Version)

Find owners who have property in **Pune city**, located in **Kothrud**, and area greater than **10000 sqft**:

SELECT DISTINCT oname

FROM Property

WHERE city = 'Pune'

AND location = 'Kothrud'

AND area_sqft > 10000;

Option 2 (View for Owner-wise property list)

CREATE VIEW OwnerWiseProperties AS

SELECT oname, pno, area_sqft, location, city

FROM Property;

Slip18

Q 2) Consider the following database(Primary keys are underlined)

Student (rno,name,city)

Teacher(tno,tname,phone_no,salary)

The relationship is as follows:

Student-Teacher: Many-to-Many with subject as a descriptive attribute.

A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostGreSQL (any 3) [6 marks]

i. List the students from “Sangamner”city.

ii. Find the maximum salary of teachers.

iii. Change the phone number of teacher ‘Dr. Mulay’ to 9834233235.

iv. List the details of the teachers in the sorted order of their name.

v. Count the number of students who have taken subject “IKS”.

C. Write a query to list the names of all students who are taught by teacher “Dr.Wani”

[4 marks]

OR

C. Create a view to list the details of students who have not taken ‘Electronics’subject.

```
-- Create Student table
CREATE TABLE Student (
  rno INT PRIMARY KEY,
  name VARCHAR(50),
  city VARCHAR(50)
);
```

```
-- Create Teacher table
CREATE TABLE Teacher (
  tno INT PRIMARY KEY,
  tname VARCHAR(50),
  phone_no VARCHAR(15),
  salary INT
);
```

```
-- Create Relationship table (M–M with Subject)
CREATE TABLE StudentTeacher (
  rno INT,
```

```
tno INT,  
subject VARCHAR(50),  
PRIMARY KEY (rno, tno, subject),  
FOREIGN KEY (rno) REFERENCES Student(rno),  
FOREIGN KEY (tno) REFERENCES Teacher(tno)  
);
```

-- Insert Students

```
INSERT INTO Student VALUES (1, 'Amit', 'Sangamner');  
INSERT INTO Student VALUES (2, 'Sneha', 'Pune');  
INSERT INTO Student VALUES (3, 'Ravi', 'Nashik');  
INSERT INTO Student VALUES (4, 'Meera', 'Sangamner');
```

-- Insert Teachers

```
INSERT INTO Teacher VALUES (101, 'Dr. Mulay', '9876543210', 50000);  
INSERT INTO Teacher VALUES (102, 'Dr. Wani', '9123456789', 60000);  
INSERT INTO Teacher VALUES (103, 'Dr. Patil', '9988776655', 45000);
```

-- Insert Relationship (Student–Teacher–Subject)

```
INSERT INTO StudentTeacher VALUES (1, 101, 'Maths');  
INSERT INTO StudentTeacher VALUES (2, 102, 'IKS');  
INSERT INTO StudentTeacher VALUES (3, 101, 'Electronics');  
INSERT INTO StudentTeacher VALUES (4, 102, 'IKS');  
INSERT INTO StudentTeacher VALUES (2, 103, 'Physics');
```

B. Queries (Any 3)

i. List the students from “Sangamner” city

```
SELECT * FROM Student  
WHERE city = 'Sangamner';
```

ii. Find the maximum salary of teachers

```
SELECT MAX(salary) AS max_salary  
FROM Teacher;
```

iii. Change the phone number of teacher ‘Dr. Mulay’ to 9834233235

```
UPDATE Teacher  
SET phone_no = '9834233235'
```

WHERE tname = 'Dr. Mulay';

iv. List details of teachers in sorted order of their name

```
SELECT * FROM Teacher
ORDER BY tname;
```

v. Count the number of students who have taken subject “IKS”

```
SELECT COUNT(DISTINCT rno) AS student_count
FROM StudentTeacher
WHERE subject = 'IKS';
```

C. Two Options

Option 1: List the names of all students taught by “Dr. Wani”

```
SELECT s.name
FROM Student s
JOIN StudentTeacher st ON s.rno = st.rno
JOIN Teacher t ON st.tno = t.tno
WHERE t.tname = 'Dr. Wani';
```

Option 2: View for students who have not taken ‘Electronics’

```
CREATE VIEW NoElectronicsStudents AS
SELECT s.rno, s.name, s.city
FROM Student s
WHERE s.rno NOT IN (
    SELECT rno FROM StudentTeacher
    WHERE subject = 'Electronics'
);
```

Slip 19

Q 2) Create the following tables (primary keys are underlined).

Order(orderno, s_order_date, amount)

Client(cno, name, address)

A client can give one or more orders, but an order belongs to exactly one

client.

Constraint: Order amount should be > 0

- A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]**
- B. Execute the following queries in PostgreSQL. (Any 3) [6marks]**
- i. Change order date of client number '04' to '12/4/19'.**
 - ii. Display all orders given in the month of October.**
 - iii. Display all orders in descending order of amount.**
 - iv. Delete all the clients of Mumbai.**
 - v. Count the number of orders having amount greater than 40000.**
- C. Write a query to display the client name who has placed maximum number of orders. [4 marks]**

OR

- C. Create a view to list names of clients having more than two sales orders.**

-- Create Client table

```
CREATE TABLE Client (  
    cno INT PRIMARY KEY,  
    name VARCHAR(50),  
    address VARCHAR(50)  
);
```

-- Create Order table

```
CREATE TABLE "Order" (  
    orderno INT PRIMARY KEY,  
    s_order_date DATE,  
    amount NUMERIC CHECK (amount > 0),  
    cno INT,  
    FOREIGN KEY (cno) REFERENCES Client(cno)  
);
```

-- Insert sample Clients

```
INSERT INTO Client VALUES (1, 'Amit', 'Mumbai');  
INSERT INTO Client VALUES (2, 'Ravi', 'Pune');  
INSERT INTO Client VALUES (3, 'Sneha', 'Delhi');  
INSERT INTO Client VALUES (4, 'Meera', 'Mumbai');  
INSERT INTO Client VALUES (5, 'Sohan', 'Nagpur');
```

-- Insert sample Orders

```
INSERT INTO "Order" VALUES (101, '2025-10-05', 50000, 1);  
INSERT INTO "Order" VALUES (102, '2025-10-12', 35000, 2);  
INSERT INTO "Order" VALUES (103, '2025-09-15', 45000, 3);
```



```
INSERT INTO "Order" VALUES (104, '2025-10-20', 25000, 4);  
INSERT INTO "Order" VALUES (105, '2025-08-10', 60000, 2);  
INSERT INTO "Order" VALUES (106, '2025-10-25', 40000, 5);  
INSERT INTO "Order" VALUES (107, '2025-10-18', 70000, 4);
```

B. Queries (Any 3)

i. Change order date of client number '04' to '12/4/19'

```
UPDATE "Order"  
SET s_order_date = '2019-04-12'  
WHERE cno = 4;
```

ii. Display all orders given in the month of October

```
SELECT * FROM "Order"  
WHERE EXTRACT(MONTH FROM s_order_date) = 10;
```

iii. Display all orders in descending order of amount

```
SELECT * FROM "Order"  
ORDER BY amount DESC;
```

iv. Delete all the clients of Mumbai

```
DELETE FROM Client  
WHERE address = 'Mumbai';
```

v. Count the number of orders having amount greater than 40000

```
SELECT COUNT(*) AS order_count  
FROM "Order"  
WHERE amount > 40000;
```

C. Two Options

Option 1: Display client name who has placed maximum number of orders

```
SELECT c.name
FROM Client c
JOIN "Order" o ON c.cno = o.cno
GROUP BY c.name
ORDER BY COUNT(o.orderno) DESC
LIMIT 1;
```

Option 2: Create a view for clients having more than two sales orders

```
CREATE VIEW ClientsWithMoreThanTwoOrders AS
SELECT c.name, COUNT(o.orderno) AS total_orders
FROM Client c
JOIN "Order" o ON c.cno = o.cno
GROUP BY c.name
HAVING COUNT(o.orderno) > 2;
```

Slip 20

Q 2) Consider the following database (Primary keys are underlined)

Customer (cust_no, cust_name, cust_city)

Account (acc_no, acc_type , balance)

Account type can be either "Saving" or "Current"

There exists a One-to-Many relationship between Customer and Account.

A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]

B .Execute the following queries in PostgreSQL (any 3) [6 marks]

i. Find the names of all Customers staying in "Delhi".

ii. Give the total balance of all Saving accounts.

iii. Find the maximum balance of an account.

iv. Increase the balance of account no 101123 by Rs 10,000/-

v. Add column "AADHAR number" to customer table.

C. Write a query to find the names and phone numbers of all Customers of

Current

accounts, having balance greater than 10,00,000/-

OR

C. Create a View to list details of all customers having saving account and not staying in city "Ahilyanagar".

```
-- Create Customer table
CREATE TABLE Customer (
```

```

    cust_no INT PRIMARY KEY,
    cust_name VARCHAR(50),
    cust_city VARCHAR(50)
);

-- Create Account table
CREATE TABLE Account (
    acc_no INT PRIMARY KEY,
    acc_type VARCHAR(10) CHECK (acc_type IN ('Saving','Current')),
    balance NUMERIC CHECK (balance >= 0),
    cust_no INT,
    FOREIGN KEY (cust_no) REFERENCES Customer(cust_no)
);

-- Insert sample Customers
INSERT INTO Customer VALUES (1, 'Amit', 'Delhi');
INSERT INTO Customer VALUES (2, 'Ravi', 'Mumbai');
INSERT INTO Customer VALUES (3, 'Sneha', 'Delhi');
INSERT INTO Customer VALUES (4, 'Meera', 'Ahilyanagar');

-- Insert sample Accounts
INSERT INTO Account VALUES (101123, 'Saving', 500000, 1);
INSERT INTO Account VALUES (101124, 'Current', 1500000, 2);
INSERT INTO Account VALUES (101125, 'Saving', 250000, 3);
INSERT INTO Account VALUES (101126, 'Current', 1200000, 4);
INSERT INTO Account VALUES (101127, 'Saving', 700000, 2);

```

B. Queries (Any 3)

i. Find the names of all Customers staying in “Delhi”

```

SELECT cust_name FROM Customer
WHERE cust_city = 'Delhi';

```

ii. Give the total balance of all Saving accounts

```

SELECT SUM(balance) AS total_saving_balance
FROM Account
WHERE acc_type = 'Saving';

```

iii. Find the maximum balance of an account

```
SELECT MAX(balance) AS max_balance  
FROM Account;
```

iv. Increase the balance of account no 101123 by Rs 10,000/-

```
UPDATE Account  
SET balance = balance + 10000  
WHERE acc_no = 101123;
```

v. Add column "AADHAR number" to customer table

```
ALTER TABLE Customer  
ADD COLUMN aadhar_no VARCHAR(12);
```

C. Two Options

Option 1: Customers of Current accounts with balance > 10,00,000

```
SELECT c.cust_name, c.cust_city  
FROM Customer c  
JOIN Account a ON c.cust_no = a.cust_no  
WHERE a.acc_type = 'Current' AND a.balance > 1000000;
```

Option 2: Create a view for customers having Saving account and not in "Ahilyanagar"

```
CREATE VIEW SavingCustomers AS  
SELECT DISTINCT c.cust_no, c.cust_name, c.cust_city  
FROM Customer c  
JOIN Account a ON c.cust_no = a.cust_no  
WHERE a.acc_type = 'Saving'  
AND c.cust_city <> 'Ahilyanagar';
```

Slip21

Q 2) Consider the following database (Primary keys are underlined)

Bus (bno, capacity, depot_name)

Driver (dno, driver_name, license_no, address, age, salary)

The relationship is as follows:

Bus and Driver are related with Many to Many relationships with the descriptive attribute Date_of_duty.

- A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]**
- B. Execute the following queries in PostgreSQL (any 3) [6 marks]**
 - i. Find the number of buses having capacity more than 20.**
 - ii. Count number of drivers who did duty on “15/01/2017”**
 - iii. Display the license number of all drivers whose name starts with ‘S’**
 - iv. Calculate the average salary of a driver.**
 - v. Find all details of driver named “Swapnil”**
- C. Write a query to find the names of all drivers who were driving buses from “Kothrud Depot” on 12/2/2009. [4 marks]**

OR

- C. Create a view to find all details of buses which are driven by drivers whose age is less than 25 years.**

```
-- Create Bus table
CREATE TABLE Bus (
    bno INT PRIMARY KEY,
    capacity INT,
    depot_name VARCHAR(50)
);
```

```
-- Create Driver table
CREATE TABLE Driver (
    dno INT PRIMARY KEY,
    driver_name VARCHAR(50),
    license_no VARCHAR(20),
    address VARCHAR(50),
    age INT,
    salary NUMERIC
);
```

```
-- Create Junction table with descriptive attribute Date_of_duty
CREATE TABLE BusDriver (
    bno INT,
    dno INT,
    date_of_duty DATE,
    PRIMARY KEY (bno, dno, date_of_duty),
    FOREIGN KEY (bno) REFERENCES Bus(bno),
    FOREIGN KEY (dno) REFERENCES Driver(dno)
```

);

-- Insert sample Buses

INSERT INTO Bus VALUES (101, 30, 'Kothrud Depot');

INSERT INTO Bus VALUES (102, 40, 'Swargate Depot');

INSERT INTO Bus VALUES (103, 25, 'Kothrud Depot');

-- Insert sample Drivers

INSERT INTO Driver VALUES (1, 'Swapnil', 'LIC123', 'Pune', 28, 30000);

INSERT INTO Driver VALUES (2, 'Sohan', 'LIC124', 'Mumbai', 24, 28000);

INSERT INTO Driver VALUES (3, 'Sunil', 'LIC125', 'Pune', 26, 32000);

-- Insert sample duties

INSERT INTO BusDriver VALUES (101, 1, '2009-02-12');

INSERT INTO BusDriver VALUES (101, 2, '2009-02-12');

INSERT INTO BusDriver VALUES (102, 3, '2017-01-15');

INSERT INTO BusDriver VALUES (103, 2, '2017-01-15');

INSERT INTO BusDriver VALUES (103, 1, '2009-02-12');

B. Queries (Any 3)

i. Number of buses having capacity more than 20

```
SELECT COUNT(*) AS bus_count
FROM Bus
WHERE capacity > 20;
```

ii. Count number of drivers who did duty on “15/01/2017”

```
SELECT COUNT(DISTINCT dno) AS driver_count
FROM BusDriver
WHERE date_of_duty = '2017-01-15';
```

iii. Display license number of all drivers whose name starts with ‘S’

```
SELECT license_no
FROM Driver
WHERE driver_name LIKE 'S%';
```

iv. Calculate average salary of a driver

```
SELECT AVG(salary) AS avg_salary  
FROM Driver;
```

v. Find all details of driver named "Swapnil"

```
SELECT *  
FROM Driver  
WHERE driver_name = 'Swapnil';
```

C. Two Options

Option 1: Names of all drivers driving buses from "Kothrud Depot" on 12/2/2009

```
SELECT DISTINCT d.driver_name  
FROM Driver d  
JOIN BusDriver bd ON d.dno = bd.dno  
JOIN Bus b ON bd.bno = b.bno  
WHERE b.depot_name = 'Kothrud Depot'  
AND bd.date_of_duty = '2009-02-12';
```

Option 2: View for all buses driven by drivers aged less than 25 years

```
CREATE VIEW YoungDriverBuses AS  
SELECT DISTINCT b.*  
FROM Bus b  
JOIN BusDriver bd ON b.bno = bd.bno  
JOIN Driver d ON bd.dno = d.dno  
WHERE d.age < 25;
```

OR

```
CREATE VIEW YoungDriverBuses AS  
SELECT *  
FROM Bus  
WHERE bno IN (  
    SELECT bno  
    FROM BusDriver bd  
    JOIN Driver d ON bd.dno = d.dno  
    WHERE d.age < 25);
```

WHERE d.age < 25
);

Slip 22

Q 2) Consider the following database(Primary keys are underlined)

Employee(eno,ename,designation,salary)

Department(dno,dname,location)

The relationship is as follows:

Employee-Department: One to Many.

Constraint: employee salary greater than 5000

A.Create the above database in PostGreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostGreSQL (any 3) [6 marks]

i. Change designation of employee “ Sagar” to Manager.

ii. Display the minimum salary of an employee.

iii. List the details of all the departments located at Nanded city.

iv. Display the details of employees whose names starts with A.

v. List the department number wise count of employees.

C.Write a query to list the department name-wise total salary. [4 marks]

OR

C. Create a view to display the department details having maximum employees.

-- Create Department table

```
CREATE TABLE Department (  
  dno INT PRIMARY KEY,  
  dname VARCHAR(50),  
  location VARCHAR(50)  
);
```

-- Create Employee table (One department → Many employees)

```
CREATE TABLE Employee (  
  eno INT PRIMARY KEY,  
  ename VARCHAR(50),  
  designation VARCHAR(50),  
  salary NUMERIC CHECK (salary > 5000),  
  dno INT,  
  FOREIGN KEY (dno) REFERENCES Department(dno)  
);
```

-- Insert Departments

```
INSERT INTO Department VALUES (1, 'HR', 'Nanded');  
INSERT INTO Department VALUES (2, 'Finance', 'Pune');
```



```
INSERT INTO Department VALUES (3, 'IT', 'Mumbai');
```

-- Insert Employees

```
INSERT INTO Employee VALUES (101, 'Sagar', 'Clerk', 12000, 1);  
INSERT INTO Employee VALUES (102, 'Amit', 'Analyst', 15000, 2);  
INSERT INTO Employee VALUES (103, 'Anil', 'Developer', 18000, 3);  
INSERT INTO Employee VALUES (104, 'Meera', 'Clerk', 10000, 1);  
INSERT INTO Employee VALUES (105, 'Ravi', 'Analyst', 16000, 2);
```

B. Queries (Any 3)

i. Change designation of employee “Sagar” to Manager

```
UPDATE Employee  
SET designation = 'Manager'  
WHERE ename = 'Sagar';
```

ii. Display the minimum salary of an employee

```
SELECT MIN(salary) AS min_salary  
FROM Employee;
```

iii. List details of all departments located at Nanded

```
SELECT *  
FROM Department  
WHERE location = 'Nanded';
```

iv. Display details of employees whose names start with A

```
SELECT *  
FROM Employee  
WHERE ename LIKE 'A%';
```

v. List department number wise count of employees

```
SELECT dno, COUNT(*) AS employee_count  
FROM Employee
```

GROUP BY dno
ORDER BY dno;

C. Two Options

Option 1: Department name-wise total salary

```
SELECT d.dname, SUM(e.salary) AS total_salary
FROM Department d
JOIN Employee e ON d.dno = e.dno
GROUP BY d.dname;
```

Option 2: Create a view for department details having maximum employees

```
CREATE VIEW DeptWithMaxEmployees AS
SELECT d.*
FROM Department d
WHERE d.dno = (
    SELECT dno
    FROM Employee
    GROUP BY dno
    ORDER BY COUNT(*) DESC
    LIMIT 1
);
```

Slip23

Q 2) Consider the following database (Primary keys are underlined)

Customer (cno, cust_name , cust_city)

Account (ano, acc_type , balance)

There exists a One to Many relationships between Customer and Account.

Constraint: Account type Should be either “Saving” or “Current”

A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostgreSQL (any 3) [6 marks]

- i. Count the number of customers whose name contains substring “in”.**
- ii. Give the average balance of all Current accounts.**
- iii. Display details of accounts in sorted order of balance.**
- iv. Increase the balance of all accounts of customer named as “Sahil” by Rs 10000.**
- v. Add column “phone_no” to customer table.**

C. Write a query to list the city-wise customer names having minimum balance. [4 marks]

OR

C. Create a view to list details of customers in Pune having maximum balance in saving account.

-- Create Customer table

```
CREATE TABLE Customer (  
    cno INT PRIMARY KEY,  
    cust_name VARCHAR(50),  
    cust_city VARCHAR(50)  
);
```

-- Create Account table (One customer → Many accounts)

```
CREATE TABLE Account (  
    ano INT PRIMARY KEY,  
    acc_type VARCHAR(10) CHECK (acc_type IN ('Saving','Current')),  
    balance NUMERIC CHECK (balance >= 0),  
    cno INT,  
    FOREIGN KEY (cno) REFERENCES Customer(cno)  
);
```

-- Insert sample Customers

```
INSERT INTO Customer VALUES (1, 'Sahil', 'Pune');  
INSERT INTO Customer VALUES (2, 'Rina', 'Mumbai');  
INSERT INTO Customer VALUES (3, 'Kiran', 'Pune');  
INSERT INTO Customer VALUES (4, 'Meera', 'Delhi');  
INSERT INTO Customer VALUES (5, 'Ankit', 'Pune');
```

-- Insert sample Accounts

```
INSERT INTO Account VALUES (101, 'Saving', 50000, 1);  
INSERT INTO Account VALUES (102, 'Current', 150000, 2);  
INSERT INTO Account VALUES (103, 'Saving', 25000, 3);  
INSERT INTO Account VALUES (104, 'Current', 120000, 4);  
INSERT INTO Account VALUES (105, 'Saving', 70000, 5);  
INSERT INTO Account VALUES (106, 'Current', 200000, 1);
```

i. Count the number of customers whose name contains substring "in"

```
SELECT COUNT(*) AS customer_count  
FROM Customer
```

```
WHERE cust_name LIKE '%in%';
```

ii. Give the average balance of all Current accounts

```
SELECT AVG(balance) AS avg_current_balance  
FROM Account  
WHERE acc_type = 'Current';
```

iii. Display details of accounts in sorted order of balance

```
SELECT *  
FROM Account  
ORDER BY balance;
```

iv. Increase the balance of all accounts of customer named “Sahil” by Rs 10000

```
UPDATE Account  
SET balance = balance + 10000  
WHERE cno = (  
    SELECT cno FROM Customer WHERE cust_name = 'Sahil'  
);
```

v. Add column “phone_no” to customer table

```
ALTER TABLE Customer  
ADD COLUMN phone_no VARCHAR(15);
```

Option 1: City-wise customer names having minimum balance (easy version)

```
SELECT c.cust_city, c.cust_name, a.balance  
FROM Customer c  
JOIN Account a ON c.cno = a.cno  
WHERE a.balance = (  
    SELECT MIN(a2.balance)  
    FROM Account a2  
    JOIN Customer c2 ON a2.cno = c2.cno  
    WHERE c2.cust_city = c.cust_city
```

);

Option 2: View for customers in Pune with maximum Saving balance (easy version)

```
CREATE VIEW PuneMaxSaving AS
SELECT c.cust_name, c.cust_city, a.balance
FROM Customer c
JOIN Account a ON c.cno = a.cno
WHERE c.cust_city = 'Pune' AND a.acc_type = 'Saving'
AND a.balance = (
    SELECT MAX(balance)
    FROM Account a2
    JOIN Customer c2 ON a2.cno = c2.cno
    WHERE c2.cust_city = 'Pune' AND a2.acc_type = 'Saving'
);
```

Slip24

Q 2) Consider the following database (primary keys are underlined).

Bus (Busno , capacity ,depot_name)

Route (Routeno ,source ,destination ,no_of_stations)

Relationship :

Bus-Route : Many to One.

Constraint: Bus capacity should not be null.

A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostgreSQL (any 3) [6 marks]

- i. Count total buses at depot “Sangamner”.**
- ii. Delete all buses having capacity less than 20.**
- iii. List all buses on route number 41.**
- iv. Find the maximum number of stations.**
- v. List all routes from “Pimpri” to “Vimannagar”.**

C. Write a query to delete all the buses of destination “Baramati”. [4 marks]

OR

C. Create a view to find the route having maximum number of stations and the buses which run on that route.

```

-- Create Route table
CREATE TABLE Route (
    Routeno INT PRIMARY KEY,
    source VARCHAR(50),
    destination VARCHAR(50),
    no_of_stations INT
);

-- Create Bus table
CREATE TABLE Bus (
    Busno INT PRIMARY KEY,
    capacity INT NOT NULL,
    depot_name VARCHAR(50),
    Routeno INT,
    FOREIGN KEY (Routeno) REFERENCES Route(Routeno)
);

-- Insert Routes
INSERT INTO Route VALUES (41, 'Pimpri', 'Vimannagar', 12);
INSERT INTO Route VALUES (42, 'Sangamner', 'Baramati', 8);
INSERT INTO Route VALUES (43, 'Pune', 'Swargate', 15);

-- Insert Buses
INSERT INTO Bus VALUES (101, 30, 'Sangamner', 41);
INSERT INTO Bus VALUES (102, 15, 'Sangamner', 42);
INSERT INTO Bus VALUES (103, 25, 'Pune Depot', 41);
INSERT INTO Bus VALUES (104, 40, 'Sangamner', 43);
INSERT INTO Bus VALUES (105, 18, 'Sangamner', 42);

```

B. Queries (Any 3)

i. Count total buses at depot “Sangamner”

```

SELECT COUNT(*) AS total_buses
FROM Bus
WHERE depot_name = 'Sangamner';

```

ii. Delete all buses having capacity less than 20

```

DELETE FROM Bus
WHERE capacity < 20;

```

iii. List all buses on route number 41

```
SELECT *  
FROM Bus  
WHERE Routeno = 41;
```

iv. Find the maximum number of stations

```
SELECT MAX(no_of_stations) AS max_stations  
FROM Route;
```

v. List all routes from “Pimpri” to “Vimannagar”

```
SELECT *  
FROM Route  
WHERE source = 'Pimpri' AND destination = 'Vimannagar';
```

C. Two Options

Option 1: Delete all buses of destination “Baramati”

```
DELETE FROM Bus  
WHERE Routeno IN (  
    SELECT Routeno  
    FROM Route  
    WHERE destination = 'Baramati'  
);
```

Option 2: Create a view for route with maximum stations and its buses

```
CREATE VIEW MaxStationRouteBuses AS  
SELECT r.Routeno, r.source, r.destination, r.no_of_stations, b.Busno,  
b.capacity, b.depot_name  
FROM Route r  
JOIN Bus b ON r.Routeno = b.Routeno  
WHERE r.no_of_stations = (  
    SELECT MAX(no_of_stations)
```

FROM Route
);

Slip 25

Q2) Consider the following database (Primary keys are underlined)

Employee (empno, emp_name, city, designation, salary)

Project (projno, proj_name, status, start_date)

There exists a Many to One relationship between Employee and Project.

Constraint: status Should be “Complete”, “In progress”.

A Create the above database in PostGreSQL and insert sufficient records. [5 marks]

B Execute the following queries in PostGreSQL (any 3) [6 marks]

i. Find the names of all Employees working on project no 105.

ii. Count the number of Projects which are “In progress”.

iii. Change start date of project “Robotics” to 15/12/2022.

iv. Increase the salaries of all employees working on project 10 by 5%.

v. Delete records of all completed projects.

C. Write a query to display the project names on which more than 3 employees are working.

[4 marks]

OR

C. Create a view to find all Employees who are working on project which started on ‘1/4/2025’ and status is “in progress”.

-- Create Project table

```
CREATE TABLE Project (  
    projno INT PRIMARY KEY,  
    proj_name VARCHAR(50),  
    status VARCHAR(20) CHECK (status IN ('Complete', 'In progress')),  
    start_date DATE  
);
```

-- Create Employee table (Many employees → One project)

```
CREATE TABLE Employee (  
    empno INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    city VARCHAR(50),  
    designation VARCHAR(50),  
    salary NUMERIC,  
    projno INT,  
    FOREIGN KEY (projno) REFERENCES Project(projno)  
);
```


-- Insert Projects

```
INSERT INTO Project VALUES (101, 'AI', 'In progress', '2025-01-01');  
INSERT INTO Project VALUES (102, 'Robotics', 'Complete', '2025-02-15');  
INSERT INTO Project VALUES (103, 'ML', 'In progress', '2025-03-01');  
INSERT INTO Project VALUES (104, 'IoT', 'Complete', '2025-01-20');  
INSERT INTO Project VALUES (105, 'Cloud', 'In progress', '2025-04-01');
```

-- Insert Employees

```
INSERT INTO Employee VALUES (1, 'Sagar', 'Pune', 'Developer', 50000,  
101);  
INSERT INTO Employee VALUES (2, 'Meera', 'Mumbai', 'Analyst', 45000,  
105);  
INSERT INTO Employee VALUES (3, 'Ravi', 'Pune', 'Tester', 40000, 105);  
INSERT INTO Employee VALUES (4, 'Sneha', 'Delhi', 'Developer', 55000,  
103);  
INSERT INTO Employee VALUES (5, 'Amit', 'Pune', 'Tester', 42000, 105);  
INSERT INTO Employee VALUES (6, 'Kiran', 'Mumbai', 'Developer', 48000,  
101);  
INSERT INTO Employee VALUES (7, 'Anil', 'Pune', 'Analyst', 43000, 105);
```

B. Queries (Any 3)

i. Find names of all employees working on project no 105

```
SELECT emp_name  
FROM Employee  
WHERE projno = 105;
```

ii. Count the number of projects which are "In progress"

```
SELECT COUNT(*) AS inprogress_projects  
FROM Project  
WHERE status = 'In progress';
```

iii. Change start date of project "Robotics" to 15/12/2022

```
UPDATE Project  
SET start_date = '2022-12-15'  
WHERE proj_name = 'Robotics';
```

iv. Increase the salaries of all employees working on project 10 by 5%

```
UPDATE Employee  
SET salary = salary * 1.05  
WHERE projno = 10;
```

v. Delete records of all completed projects

```
DELETE FROM Project  
WHERE status = 'Complete';
```

C. Two Options

Option 1: Display project names on which more than 3 employees are working

```
SELECT p.proj_name  
FROM Project p  
JOIN Employee e ON p.projno = e.projno  
GROUP BY p.proj_name  
HAVING COUNT(e.empno) > 3;
```

Option 2: Create a view for employees working on projects started on '1/4/2025' and status "In progress"

```
CREATE VIEW EmployeesOnApr1InProgress AS  
SELECT e.empno, e.emp_name, e.city, e.designation, e.salary, e.projno  
FROM Employee e  
JOIN Project p ON e.projno = p.projno  
WHERE p.start_date = '2025-04-01' AND p.status = 'In progress';
```